



كلية الهندسة المعلوماتية

جامعة دمشق

السنة الرابعة

مشروع البرمجة التفرعية

التقرير

الطلاب :

احمد مؤيد علوش

نوران محمد كنذر

ريما أسامة آغي

وثام وسيم حسون

رضا احمد عارف

بإشراف :

م. حذيفة عقيل

الطلب الأول :

حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

: Race Condition

في أنظمة التجارة الإلكترونية التقليدية، يتم تعديل المخزون البرمجي عبر قاعدة البيانات بالخطوات التالية: قراءة القيمة الحالية → التعديل في الذاكرة → حفظ القيمة الجديدة.

عندما يضغط مئات المستخدمين على زر "شراء" في نفس الأجزاء من الثانية لشراء منتج متبقٍ منه قطعة واحدة فقط

1. **تداخل الخيوط (Thread Overlapping):** يقرأ الخيط (Thread) الخاص

بالمستخدم الأول أن كمية المنتج الحالية هي (Stock = 1)

2. **فقدان التحديثات (Dirty Read / Lost Update):** قبل أن يقوم الخيط

الأول بحفظ التعديل، يقرأ الخيط الخاص بالمستخدم الثاني نفس القيمة

(Stock = 1).

3. **النتيجة:** ينجح كلا المستخدمين في إتمام الطلب، ويتحول المخزون إلى قيمة

سالبة أو غير منطقية، وهو ما يُعرف بـ **البيع الزائد (Overselling)** هذا

يتسبب في انهيار مصداقية النظام وتضارب البيانات المالية والمخزنية.

تم تطبيق ثلاث استراتيجيات هندسية :

التحديث الذري المباشر (Atomic Database Updates) :

بدلاً من سحب البيانات إلى الخادم وتعديلها ثم إعادتها تم الاعتماد على الـ F

expressions

في Django هذه الآلية تنقل العمل الحسابي بالكامل إلى مستوى قاعدة البيانات

مباشرة بعبارة موازية مثل:

UPDATE products SET stock = stock - quantity WHERE id = product_id

ميزة الأداء: يتم التحديث كعملية واحدة غير قابلة للتجزئة (Atomic) مما يمنع الخيوط من قراءة بيانات قديمة (Stale Data)

جيدة في العمليات السريعة والمباشرة جداً مثل التعديل على المخزون ولكن لا تعطي مرونة برمجية للتحقق من شروط معقدة في الذاكرة قبل التحديث

القفل التشاؤمي (Pessimistic Locking) :

يستخدم في العمليات المالية والمخزنية الحرجة جداً (مثل لحظة إنشاء الطلب الفعلي والدفع من المحفظة) تم استخدام الدالة `select_for_update()` لإطلاق أمر الحجز الصريح **SELECT ... FOR UPDATE** على مستوى قاعدة البيانات.

يتم حجز وقفل السطر (Row-level Lock) الخاص بالمنتج أو محفظة المستخدم فوراً، ويجبر أي خيط آخر يحاول الوصول لنفس المورد على الانتظار في طابور قاعدة البيانات حتى تنتهي المعاملة الحالية (Transaction) تماماً فهو يضمن حماية مطلقة للبيانات من أول محاولة لأن الخيوط تنتظر دورها في طابور منظم على مستوى قاعدة البيانات

يقلل من سرعة الاستجابة الإجمالية للنظام (Throughput) إذا طال القفل، وقد يسبب بطء في النظام

القفل المتفائل (Optimistic Locking) :

يفترض أن التضارب نادر الحدوث. تم دمج حقل مخصص للإصدار (version) في نموذج المنتج عند التحديث يتم التحقق من أن الإصدار لم يتغير أثناء فترة المعالجة

WHERE id = product_id AND version = current_version

في حال فشل التحديث بسبب تغير رقم الإصدار من قبل مستخدم آخر، لا ينهار النظام، بل يتم تطبيق خوارزمية تراجع عشوائي ذكي (Smart Random Backoff) عبر

time.sleep(random.uniform(0.01, 0.03))

لإعادة المحاولة حتى 3 مرات، مما يقلل من احتمالية حدوث اختناق (Lock Contention).

ولكن تحت الضغط الشديد جداً (High Contention) على منتج واحد مثل عروض Black Friday أو قطعة أخيرة من منتج تريد ستفشل المحاولات الـ 3 (Max 3 Retries) بسبب كثرة التضاربات مما يضطر النظام لرمي استثناء (Exception) للمستخدم ويقلل من تجربة المستخدم (UX)

الطلب الثاني :

إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

اختناق الخادم واستنزاف الذاكرة (Memory & DB Exhaustion) :

في الأنظمة العشوائية التي لا تفرض قيوداً على مواردها يؤدي تعرض النظام لضغط مفاجئ (مثل طلبات متزامنة من مئات المستخدمين) إلى :

استنزاف اتصالات قاعدة البيانات: (Connection Pool Exhaustion) يقوم الخادم بفتح إغلاق صريح لكل عملية طلب برمجية (Request) مع قاعدة البيانات إنشاء اتصال جديد هو عملية مكلفة جداً حاسوبياً وتستهلك وقتاً وذاكرة وتحت الضغط تنهار قاعدة البيانات وتطلق استثناء الذاكرة الشهير. (Too many connections)

انفجار الذاكرة بسبب تضخم البيانات: (Memory Bloat) عند طلب تصفح المنتجات (التي قد تصل لآلاف الأسطر)، يقرأ الخادم كامل البيانات ويحولها إلى

كائنات لزوجها في الذاكرة العشوائية (RAM) دفعة واحدة لإرسالها، مما يسبب خروج السيرفر عن الخدمة فوراً نتيجة نفاد الذاكرة.(Out of Memory)

انهيار المعالج بسبب المهام المتراكمة: معالجة المهام الثقيلة (مثل إصدار التقارير أو إرسال الإشعارات) في مسار الطلب الرئيسي تستنزف خيوط المعالج وتجعل زمن استجابة النظام للمستخدمين الآخرين شبه منعدم.

التقنيات الهندسية المعتمدة :

استدامة اتصالات قاعدة البيانات وتقييد عمرها (CONN_MAX_AGE = 60) :

بدلاً من تدمير الاتصال بعد كل طلب وإعادة بنائه تم ضبط الخيار CONN_MAX_AGE على 60 ثانية هذا يعني الحفاظ على استمرارية الاتصال (Persistent Connections) وإعادة استخدامه للطلبات القادمة خلال هذه الدقيقة هذا يؤدي إلى تخفيف الحمل الحسابي على المعالج والذاكرة في ال PostgreSQL بنسبة ضخمة ومنع استنزاف بركة الاتصالات تحت الضغط العالي.

التحكم في ذاكرة ال RAM وحجم الشبكة عبر ال Pagination :

حصر سحب البيانات من قاعدة البيانات بـ 10 منتجات فقط لكل طلب. هذا يحمي ذاكرة الخادم العشوائية (RAM) من التضخم (Memory Bloating) ويجعل حجم حزم البيانات المرسلة عبر الشبكة (Payload) خفيفاً وثابتاً دائماً، مما يحسن زمن الاستجابة (Latency)

تحكم بالطاقة الاستيعابية لل Workers المسؤولين عن المهام الخلفية غير المتزامنة:

(CELERY_WORKER_CONCURRENCY = 4)

تقييد عدد العمليات المتوازية التي ينفذها ال Worker بـ 4 مهام كحد أقصى موازٍ لأنوية المعالج المتاحة، لمنع استهلاك السيرفر بالكامل.

(CELERY_WORKER_PREFETCH_MULTIPLIER = 1)

هذا يمنع ال Workers من سحب مهام كثيرة من الطابور Queue واحتكارها في الذاكرة دون تنفيذ مما يضمن توزيعاً عادلاً للمهام Fair Distribution وعدم اختناق ال Workers تحت الضغط الشديد

حماية الذاكرة المؤقتة من السقوط (SOCKET_TIMEOUT = 2) :

وضع قيود زمنية صارمة للاتصال بRedis إذا واجه سيرفر الكاش (Redis) ضغطاً كبيراً أو تأخراً مما يتسبب بوقوف الخيوط وحجزها للذاكرة فلن ينتظر السيرفر طويلاً بل يقطع الاتصال بعد ثانيتين ويتابع القراءة من قاعدة البيانات

إدارة نموذج الخيوط (Thread Pool Management) عبر Waitress :

في بيئة الإنتاج والتشغيل الفعلي لا يعمل Django بمفرده لاستقبال الطلبات بل يتم تخديمه بواسطة خادم ويب إنتاجي (Production WSGI Server) مثل Waitress عبر سطر الأوامر

```
waitress-serve --port=8000 my_site.wsgi:application
```

يتم التحكم في الطاقة الاستيعابية وحماية الموارد :

حوض الخيوط المخصص (Worker Thread Pool) يقوم خادم Waitress عند الإقلاع بإنشاء قنوات معالجة مستقلة متوازية. (Multi-threaded Architecture) بدلاً من استهلاك الذاكرة عبر فتح عمليات (Processes) منفصلة بالكامل يتم الاعتماد على الخيوط (Threads) التي تشارك نفس مساحة الذاكرة للتطبيق.

إدارة كفاءة الانتظار (I/O Wait Efficiency) استخدام الخيوط يعتبر الحل الأمثل للعمليات التي تتطلب انتظاراً طويلاً لمدخلات ومخرجات البيانات (I/O Bound Tasks)، مثل فترات استعلام واسترجاع البيانات من قاعدة بيانات PostgreSQL حيث يمكن للخيوط الأخرى متابعة تخدم الزبائن دون تجميد النظام.

منه يتم تحديد سقف ثابت لحوض الخيوط يخلق ما يسمى بـ **Fixed Thread Pool** مما يضمن تقييد استهلاك وحدة المعالجة المركزية (CPU) والذاكرة (RAM) بحدود آمنة ومعروفة مسبقاً، مهما تصاعد عدد المستخدمين المتزامنين على النظام

نظام طوابير الانتظار (Request Queuing) :

يعمل هذا الطابور كـ (**Buffer Area**) حيث تحتفظ قاعدة النظام بالطلبات لفترة وجيزة جداً حتى يتحرر أول خيط معالجة داخل حوض التطبيق ليقوم بأخذ الطلب اللي بعده هذا الإجراء يمنع انهيار السيرفر ويحد من ظاهرة عبء تبديل السياق المفرط (**Context Switching Overhead**) التي تحدث عندما يحاول المعالج التبديل العشوائي والمتزامن بين آلاف المهام في نفس الوقت بشكل افتراضي، خادم Waitress يمتلك إعداداً داخلياً مخفياً اسمه backlog وقيمته الافتراضية هي 1024. إذا أردت إظهارها للدكتور بشكل صريح وتعديلها، يمكنك كتابتها مباشرة في سطر تشغيل السيرفر بالTerminal

```
waitress-serve --port=8000 --backlog=60 my_site.wsgi.application
```

في هذا الطلب تم تحديد 60 طلب كحد أقصى

محددات المعدل (Rate Limiting / Throttling) :

لحماية حوض الخيوط المشترك (Thread Pool) وقاعدة البيانات من خطر الاستنزاف المتعمد الناتجة عن الهجمات الإلكترونية (مثل هجمات الإغراق والدخول المتكرر الضار)، أو حتى نتيجة السلوك العشوائي المتكرر من قبل مستخدم واحد، تم تضمين سياسة التقييد برمجياً داخل واجهة التطبيق (Django REST Framework) قيود صارمة تحدد سقف العمليات المسموح بها لكل عنوان IP أو مستخدم مجهول أو مسجل خلال وحدة زمنية معينة لضمان التوزيع العادل للموارد المتاحة بين كافة الطلبات الحقيقية بالتساوي (Fairness)، ومكافحة حدوث حالة حرمان الموارد الحاد (**Resource Starvation**) للمستخدمين الآخرين التي قد تتسبب بخروج النظام عن الاستجابة الفعالة

الطلب الثالث :

المعالجة غير المتزامنة (Asynchronous Queues) :

تعتمد البنية الهيكلية للنظام على فصل العمليات المرتبطة بطلب ال HTTP الفوري عن العمليات الثقيلة حسابياً أو المرتبطة بالشبكة الخارجية، وذلك بهدف الحفاظ على زمن استجابة قليل جداً للمستخدمين Low Latency

تم تحقيق ذلك عبر دمج **Celery** ك Task Runner مع **Redis** ك Message Broker

عزل المهام اللحظية الخارجية

مثل إرسال رسالة إيميل يتم تغليف تابع إرسال البريد الإلكتروني للمستهلك باستخدام المزخرف `@shared_task` بمجرد إتمام الشراء يضع المهمة في طابور Redis ويرد فوراً للمستخدم بـ "تم استلام طلبك"، ليتولى عمال Celery إرسال البريد بالخلفية.

او ارسال اشعارات بشكل عام تتم المهمة بنفس الطريقة .

الطلب الرابع :

معالجة البيانات الضخمة على دفعات (Batch Processing) :

في حال طلب تقرير جرد للمخزون أو حساب مبيعات يضم 50,000 منتج أو فاتورة بالطريقة التقليدية عبر استخدام `Product.objects.all()`

تضخم الذاكرة: (Memory Bloat) سيقوم بايثون بسحب كافة هذه السجلات وتحويلها إلى كائنات Python Objects وتخزينها في ال RAM دفعة واحدة. هذا يسبب قفزة حادة ومفاجئة في استهلاك الذاكرة قد تؤدي إلى انهيار السيرفر بالكامل بسبب تخطي الحد المسموح به. (Out of Memory - OOM)

قفل الجداول (Table Locking): الاستعلام الحجمي يضع ضغطاً هائلاً على محرك قاعدة البيانات ويحجز جدول المنتجات أو الطلبات لفترة طويلة، مما يمنع الزبائن الآخرين من إتمام عمليات الشراء اللحظية ويتسبب بظهور خطأ مهلة الانتظار (Gateway Timeout).

لتفادي هذه المشاكل يعتمد النظام على استراتيجية المعالجة على دفعات (Batch Processing) عبر وظائف خلفية (Background Jobs) مجدولة ومجزأة تضمن تفتيت البيانات الضخمة إلى كتل صغيرة وعمليات (Chunks).

آلية الـ Chunk Processing :

باستخدام دالة `iterator(chunk_size=500)`. نجبر النظام على فتح "مجرى بيانات مستمر (Stream)" مع قاعدة بيانات PostgreSQL. يتم جلب 500 منتج فقط إلى الذاكرة، معالجتها وكتابتها في ملف CSV، ومن ثم إفراغ الذاكرة فوراً وسحب الـ 500 منتج التالية.

وبهذا يبقى خط استهلاك الذاكرة العشوائية (RAM Profile) مستقراً وثابتاً تماماً (Flat Line) سواء كان المخزون يحتوي على 500 منتج أو 500,000 منتج.

التجميع الحسابي المرحل على مستوى قاعدة البيانات :

بدلاً من جلب آلاف الفواتير اليومية إلى بيئة بايثون وعمل حلقة تكرارية (for loop) لجمع الأسعار وهو ما يستهلك المعالج والذاكرة نقوم بترحيل العبء الحسابي بالكامل عبر تعليمات `aggregate(Sum(...))` ليتم معالجته داخل نواة الـ PostgreSQL مباشرة

التحديث الجماعي للأسعار (Mass Price Updates) :

عند إطلاق أمر التعديل دفعة واحدة سيقوم السيرفر بمحاولة تعديل 50,000 سطر بلحظة زمنية واحدة داخل قاعدة البيانات هذا الإجراء يجبر محرك الـ PostgreSQL على تطبيق عملية قفل الجدول بالكامل **Exclusive Table Lock** لحماية سلامة

البيانات نتيجة لذلك أي زبون يحاول في هذه اللحظة تصفح المنتجات أو إتمام عملية شراء سيعلق طلبه (Hanging) ويواجه الموقع حالة جمود كاملة وشلل في الأداء

والحل يتم ترحيل هذه العملية الثقيلة لتتم في الخلفية عبر وظيفة مجدولة (Background Job) تقسم الـ 50,000 منتج إلى دفعات صغيرة متتالية (Chunks) بحيث يتم تعديل أسعار 500 منتج فقط في كل دفعة

منه يعطي متسعاً زمنياً ميكروبياً لمحرك قاعدة البيانات لكي "يتنفس Database" (Breathing Time) حيث يقوم بتمرير طلبات الشراء الحقيقية وفواتير الزبائن الحالية بسلسلة فائقة بين كل دفعة تعديل وأخرى، مما يضمن استمرارية الخدمة بنسبة 100% دون أن يشعر المستخدم بأي بطء في الموقع.

الطلب الخامس :

توزيع الأحمال (Load Distribution) :

عندما يتعرض النظام لذروة مفاجئة في أعداد المستخدمين (Traffic Spikes) ، فإن الاعتماد على خادم ويب واحد (Single Server) يصبح نقطة فشل مركزية (Single Point of Failure). لحماية النظام من الانهيار وتحقيق مبدأ "الإتاحة العالية High Availability"

Availability

اختناق الخادم المفرد (Server Bottleneck)

في المعماريات التقليدية، تصب جميع طلبات الـ HTTP القادمة من المتصفحات في خادم تطبيق واحد. تحت الضغط العالي يؤدي إلى استنزاف الموارد بالكامل تصل نسبة استهلاك المعالج (CPU) والذاكرة (RAM) إلى 100%، مما يتسبب ببطء قاتل في الاستجابة.

آلية موازنة الأحمال الذكية: (Load Balancing & Proxying)

لحظر هذا الاختناق، تم إدخال خادم **Nginx** كموزع أحمال أمامي ووكيل عكسي (Reverse Proxy) بدلاً من ترك طلبات الـ HTTP تصب مباشرة في خادم التطبيق الواحد، تقف بوابة Nginx في الخط الأول لتلقي كافة الطلبات وتوزيعها برمجياً على خوادم التطبيق الخلفية بالتناوب وعبر خوارزمية **Round Robin التوزيع الدائري** **بالدور** هذا التفتيت يضمن تقسيم العبء الحسابي بالتساوي بحيث لا يواجه أي خادم مفرد ضغطاً يدفعه للانهييار